SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	K Mahesh Sai	**Reg. No.:**	192412674
Section / Batch:	3rd Year	**Date:**	23-10-2026

Code of Conduct

I ~~K Mahesh Sai~~, ~~192412674~~ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: K Mahesh Sai

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q5**

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption.
- Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.

(b) Analyze the computational challenges of large game trees.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

(d) Design an evaluation function and justify the factors considered.

(e) Recommend a strategy for balancing optimality and real-time performance.

ASSESSMENT TOOL 2 – COMPLETE ANSWERS

Course: CSA17 – Artificial Intelligence | CO2 | Scenario Based Assignment

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

(a) Greedy Best-First Search: behaviour, strengths and weaknesses

Greedy Best-First Search selects the frontier node with the smallest heuristic h(n), here the straight-line distance to the medicine delivery destination. It prioritizes apparent closeness to the goal and may quickly find a route when the map and heuristic are informative.

- Strengths: often explores fewer nodes; easy to implement; useful when a quick route estimate is needed.
- Weaknesses: ignores the cost already incurred, g(n); the geometrically closest route may cross flooded roads or dangerous weather; it is not generally optimal and can be misled by inaccurate or stale information.

(b) A* search and cost–heuristic balance

A* ranks frontier nodes by f(n) = g(n) + h(n), where g(n) is the known cost from the start and h(n) estimates remaining cost. Thus it considers both the travel already paid for and the estimated distance remaining. If edge costs are non-negative and h is admissible (and implementation conditions are satisfied), A* can return an optimal path in a static model. For this drone, g should reflect time, terrain, weather and risk penalties, while h should estimate the remaining travel cost in compatible units.

(c) Impact of heuristic accuracy

A useful heuristic estimates remaining cost without systematically overestimating it when optimality is required. A close, accurate heuristic directs both methods toward promising routes. Greedy search is especially sensitive because h alone determines its choice. A* can correct for a costly route through g(n), but a poor heuristic may still cause extra expansions; an overestimating heuristic can make A* lose its optimality guarantee.

(d) Dynamic blocked paths

When a road becomes blocked, the route graph and edge costs change. A search using stale data may return an unsafe or unusable path. The drone should receive updates, mark blocked edges unavailable, update affected costs and re-plan from its current location. Repeated changes can require repeated searches, adding computation and delay. Incremental re-planning methods can reuse prior search information when the map changes slightly.

(e) Recommendation

Use A* with a risk-aware cost function, frequent map/weather updates, and re-planning whenever new information invalidates the route. It balances actual travel cost and estimated remaining cost, unlike Greedy search. Safety constraints should treat known hazards as forbidden or assign them very high cost. If strict deadlines make full A* too slow, use bounded/incremental A* and a safe fallback route; do not trade away safety merely for speed.

Question 2: Smart City Traffic Signal Optimization

(a) Optimization formulation

Represent a candidate solution as a vector of signal plans for all intersections: green durations, phase order and offsets. Define an objective such as J = w1·(total waiting time) + w2·(fuel consumption) + w3·(congestion), with non-negative weights reflecting city priorities. Minimize J subject to constraints such as minimum/maximum green times, safe phase transitions, pedestrian crossing time, and intersection capacity. Traditional exhaustive

search is inefficient because the combinations of timings across hundreds of intersections grow exponentially, and traffic conditions change over time.

(b) Hill Climbing and limitations

Hill climbing starts with a current signal plan, evaluates neighbouring plans (for example, changing one green duration slightly), and moves to a neighbour with a lower objective J. It repeats until no neighbouring plan improves the score. It is simple and memory-efficient, but it can stop at a local optimum, cannot naturally accept temporary deterioration, and may keep chasing traffic conditions that change before the plan is deployed.

(c) Local maxima, plateaus and ridges

- Local optimum: a plan improves one intersection but causes queues to spill into nearby streets; no small local change appears better even though a coordinated city-wide plan would be better.
- Plateau: several timing plans have nearly equal scores, so the algorithm has no clear direction and may stop or wander.
- Ridge: improvement requires coordinated changes across multiple intersections; changing only one signal at a time may fail to follow the narrow path of improvement.

(d) Simulated Annealing and Genetic Algorithms

Simulated Annealing usually accepts an improving move, but sometimes accepts a worse move with probability $\exp(-\Delta J/T)$, where ΔJ is the increase in objective and T is temperature. At high T it explores broadly; as T decreases it becomes more selective, helping it escape local optima early. A Genetic Algorithm maintains a population of timing plans, selects better plans, combines them through crossover, and introduces mutations. Population diversity explores multiple regions, though both methods need suitable parameters and constraint handling.

(e) Recommendation

A practical choice is a Genetic Algorithm for offline or periodic city-wide coordination, combined with fast local adjustments or rolling-horizon optimization online. Its population can explore combinations across many intersections and optimize multiple competing measures. Simulated Annealing is a simpler alternative when memory and implementation cost are priorities. In either case, enforce traffic-safety constraints and re-optimize as sensor data changes; compare candidates using realistic simulation before deployment.

Question 3: Autonomous Rover Exploring Mars

(a) Why online search is needed

An offline search agent assumes it has a sufficiently complete map/model before planning. The rover begins without a complete map and discovers terrain as it moves. An online search agent interleaves sensing, planning and acting: it chooses an action using current knowledge, observes the result, updates its model and replans. This suits unknown terrain and limited communication.

(b) Partial observability and environmental challenges

- Sensors have limited range and may not reveal hazards hidden behind rocks or inside craters.
- Measurements may be noisy, and terrain traversability can be uncertain.
- The rover has limited battery, time and computing resources; a wrong move can damage equipment or prevent sample collection.
- Communication delays prevent relying on continuous human instructions, so decisions must be made locally with safety margins.

(c) Building and updating the internal model

1. Initialize a map with known landmarks and unknown cells marked as unexplored.

2. Use cameras, range sensors and other instruments to estimate nearby obstacles, slopes and traversable surfaces.
3. Record visited locations, actions, outcomes, energy use and estimated travel costs; update confidence when new observations arrive.
4. Mark hazards or blocked regions, update the reachable map and plan a safe next step toward a sampling target.
5. Monitor localization uncertainty and battery; if an unexpected obstacle appears, stop or choose a safe alternative and replan.

(d) Online vs uninformed and informed search

Uninformed methods such as breadth-first or depth-first search do not use a goal-distance estimate; they may waste actions exploring irrelevant areas. Informed methods such as A* use a heuristic to prioritize promising routes, but typically need a known graph or model. Online search acts while learning the graph, so it must account for what is currently known and the cost of exploration. A rover can combine online exploration with a heuristic based on distance to the target and risk.

(e) Suitable approach

Use a risk-aware online agent with incremental map building, heuristic-guided exploration, obstacle avoidance and continual re-planning. Prefer safe, well-observed terrain when the risk of failure is high, and reserve battery for recovery and communication. Unknown cells should not automatically be treated as safe. This approach adapts to discoveries while keeping safety and scientific objectives explicit.

Question 4: University Exam Timetable Generation

(a) CSP formulation

Variables: one variable per exam, for example X_i for each course i. Domains: allowable time slots for each exam, optionally paired with an exam hall. Constraints: (1) exams sharing students must use different time slots; (2) each hall can host at most one exam in a slot and hall capacity must be sufficient; (3) prerequisite/order constraints require selected subjects to occur before others; (4) exams require available faculty/invigilators; and (5) accommodations require suitable rooms, time allowances or accessibility. A valid solution assigns every variable a value satisfying all hard constraints.

(b) Backtracking search

Backtracking assigns a slot to one exam at a time. After each assignment it checks constraints against assignments already made. If a conflict occurs, it rejects that value and tries another value. If no value works for the current exam, it undoes the most recent assignment and backtracks to an earlier exam. Variable-ordering heuristics, such as Minimum Remaining Values (MRV), can choose the exam with the fewest legal slots first; degree heuristic breaks ties by choosing the exam constraining the most unassigned exams.

(c) Constraint propagation and forward checking

After assigning an exam, forward checking removes incompatible values from the domains of unassigned neighbouring exams. If a domain becomes empty, the current partial timetable cannot be completed, so the solver backtracks immediately instead of exploring deeper. Propagation reduces the search tree and exposes conflicts early. Stronger consistency checks may propagate consequences beyond immediate neighbours, at additional computational cost.

(d) Challenges and scalable strategy

Real data can be large and incomplete; student registrations change, room capacities and accessibility vary, faculty availability is limited, and hard constraints can conflict. Some preferences (avoid early exams, spread workload) are soft rather than absolute. Use CSP backtracking with MRV, degree heuristic and forward checking, then optimize soft preferences with branch-and-bound or a weighted objective. Decompose by

campuses or time periods where dependencies permit, maintain clear hard/soft constraints, and re-run locally when registrations change. Always validate the final timetable against every constraint.

Question 5: Strategic Game AI for Real-Time Decision Making

(a) Minimax model

Minimax models the game as a tree of legal states and moves. MAX represents the AI and chooses the move with the highest guaranteed value; MIN represents the opponent and chooses the reply that minimizes MAX's value. At terminal or depth-limited leaves, an evaluation function estimates how favourable each state is to MAX. Values are backed up: MAX nodes take the maximum child value and MIN nodes take the minimum. The AI chooses the root move with the best backed-up value.

(b) Large game-tree challenges

If the average branching factor is b and search depth is d, a full tree can contain O(b^d) nodes. This exponential growth makes exhaustive search too slow for strict real-time limits. A depth cutoff can miss long-term consequences; poor evaluation functions can mis-rank positions; and simultaneous strategic, economic and tactical decisions increase branching. The agent must also manage memory, legal move generation and unpredictable opponent behaviour.

(c) Alpha-Beta pruning

Alpha-beta pruning gives the same minimax result while skipping branches that cannot affect the final decision. Alpha (α) is the best value MAX can guarantee so far; beta (β) is the best value MIN can guarantee so far. At a MAX node update α = max(α, child value); at a MIN node update β = min(β, child value). When α ≥ β, the remaining children of that node can be pruned because an ancestor already has an option at least as good. Good move ordering increases pruning: in ideal ordering, effective complexity can approach O(b^(d/2)) rather than O(b^d), while worst-case complexity remains O(b^d).

(d) Example evaluation function

One practical depth-limited evaluation function is:

Eval(s) = 5(MATERIAL) + 3(UNIT SAFETY) + 2(RESOURCE ADVANTAGE) + 2(OBJECTIVE CONTROL) + 1(MOBILITY)

Each feature is scored from MAX's perspective, with opponent advantage subtracted from MAX advantage. Material measures surviving unit strength; safety penalizes threatened units and rewards defensible positions; resources estimate the ability to build and sustain forces; objective control reflects map goals; and mobility rewards useful legal options. Weights should be tuned and tested against varied opponents. Terminal wins/losses should receive much larger positive/negative values than ordinary positional scores.

(e) Real-time recommendation

Use iterative-deepening minimax with alpha-beta pruning, strong move ordering and a time budget. Keep the best move from the last fully completed depth so the agent can act when time expires. Use a fast evaluation function at the depth limit, optionally extend tactically unstable states, and cache previously evaluated positions with a transposition table. This produces a good decision quickly while preserving minimax correctness for the portion of the tree searched; it does not guarantee a globally optimal move when search is cut off.

Conclusion

The scenarios show that AI search methods must be selected according to the environment and objective. A* balances route cost and heuristic guidance; stochastic or population-based local search handles large optimization spaces; online agents learn while acting in unknown environments; CSP methods enforce timetable

constraints; and minimax with alpha-beta pruning supports adversarial decisions under limited time. In all cases, reliable sensing, suitable cost/evaluation functions, constraint checking and re-planning are essential.

References / Further Reading

1. Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 4th edition, Pearson.

2. Course assessment brief: CSA17 – Artificial Intelligence, Assessment Tool 2 – Scenario Based Assignment (provided source document).